

Evidencia de aprendizaje 3 — IU Digital Radio

Estudiante: *Escribe aquí tu nombre completo*

Programa: Desarrollo de Software

Asignatura: Programación de dispositivos móviles

Modalidad: Individual

Fecha: *Escribe la fecha de entrega*

1. Descripción de la solución

IU Digital Radio es una aplicación Android nativa desarrollada en Kotlin y Jetpack Compose. Permite consultar emisoras reales mediante la API pública Radio Browser, seleccionar una estación, reproducir su stream con Media3 ExoPlayer, pausar, silenciar y capturar una fotografía para el perfil mediante la cámara del dispositivo.

2. Arquitectura y decisiones técnicas

La aplicación concentra la pantalla en `MainActivity` y separa la consulta remota en `RadioBrowserRepository`. La interfaz se construye declarativamente con Material 3 y los componentes `Column`, `Row`, `Card`, `LazyColumn` y `Modifier`. El estado `isPlaying`, `isMuted` y `selectedStationId` se mantiene con `rememberSaveable`, de modo que las interacciones principales sobreviven a cambios de configuración como la rotación. La fotografía capturada se conserva en memoria durante la sesión, ya que `TakePicturePreview` entrega un `Bitmap` temporal.

El reproductor se implementa con Media3 ExoPlayer. Cuando cambia la estación seleccionada, se crea un `MediaItem`, se prepara el stream y se sincronizan el volumen y el estado de reproducción. El código libera el reproductor al salir de la composición.

3. Permisos y hardware

El manifiesto declara `android.permission.CAMERA`, `android.permission.VIBRATE` y `android.permission.INTERNET`. La cámara se solicita en tiempo de ejecución antes de lanzar `ActivityResultContracts.TakePicturePreview`. La vibración es un permiso normal y se ejecuta con `VibratorManager` en Android 12 o superior y con `Vibrator` en versiones anteriores.

4. Evidencias visuales

Inserta aquí las capturas de pantalla con una breve descripción:

1. **Interfaz inicial:** captura de la cabecera de perfil, reproductor y catálogo.
2. **Permiso de cámara:** captura del diálogo del sistema.
3. **Foto capturada:** captura del perfil actualizado.
4. **Reproducción:** captura de la estación seleccionada y el estado “Reproduciendo en vivo”.
5. **Cambio de emisora:** captura de otra estación activa.
6. **APK:** captura de Android Studio mostrando la generación exitosa y la ruta del archivo.

5. Pruebas realizadas

Prueba	Resultado esperado	Resultado obtenido
Inicio de aplicación	Se muestra la interfaz Compose	Pendiente de completar
Solicitud de cámara	Android solicita permiso	Pendiente de completar
Captura de foto	La imagen aparece en el perfil	Pendiente de completar
Play/Pause	Cambia el estado y controla ExoPlayer	Pendiente de completar
Mute	Cambia el volumen y vibra	Pendiente de completar
Selección de estación	Actualiza la estación activa	Pendiente de completar
Rotación	Conserva estación y estados	Pendiente de completar
Compilación	Se genera <code>app-debug.apk</code>	Pendiente de completar

6. Enlaces de entrega

Repositorio GitHub/GitLab: *Pega aquí el enlace público*

Video de demostración en Google Drive: *Pega aquí el enlace con permisos de lectura*

7. Conclusión

La solución integra maquetación declarativa, estado reactivo, hardware, permisos, consumo de API y reproducción multimedia en una aplicación nativa instalable. Las emisoras provienen de Radio Browser, por lo que su disponibilidad puede cambiar como parte de la naturaleza de un servicio de terceros.

Referencias

- [Radio Browser API](#)
- [Media3 ExoPlayer](#)
- [TakePicturePreview](#)
- [Solicitud de permisos en Android](#)